

Energy-Aware Adaptive Self-Speculative Decoding

Hardware-Aware Energy-Efficient LLM Inference on Edge Devices

Project Report | Repository: *Energy-Efficiency-DL-Model*

1. Abstract

This project studies how to reduce the energy and latency of small language model (LLM) inference on laptops and consumer GPUs. It began as a plan to combine model compression, hardware-aware quantization and CALM-style early exit. It has since been refined into an energy-aware controller for self-speculative decoding, where early layers draft tokens and the full model verifies them. The repository now holds a synthetic controller demo and a real-model feasibility pilot built on SmolLM2-135M and LayerSkip Llama-3.2-1B. All pilot outputs matched native greedy decoding. No energy saving is claimed yet, because CPU energy could not be measured.

2. Introduction and Motivation

- LLM inference on edge hardware is limited by battery, memory and thermal budgets.
- Speculative decoding speeds up generation, but wasted drafts can cost more energy than they save.
- The central question is when speculation saves energy per committed token, and when ordinary decoding is safer and cheaper.

3. Objectives

- Build a controller that chooses between ordinary and speculative decoding at each step.
- Base each choice on context length, recent draft acceptance and estimated energy cost.
- Guarantee output identical to ordinary greedy decoding.
- Validate the mechanics on real models before any energy claims.

4. Methodology

4.1 Original Research Plan

The first plan was a three-stage pipeline, retained in the README for context:

- Compression: distil Qwen3-1.7B (teacher) into Qwen3-0.6B (student), then prune.
- Quantization: FP16, INT8 and INT4 with hardware-aware configuration search.
- Adaptive early exit: CALM-style confidence check at each layer.

Only the distillation script (`edge-llm-inference/distill.py`) was implemented. It uses 200 Alpaca examples, temperature 2.0 and a 0.5 weighting between distillation and language-model loss.

4.2 Energy-Aware Controller (Synthetic Prototype)

- Offline preparation: profile actions such as ordinary decoding, draft depth 4 or 8, and draft length 2, 3 or 5.
- Online inference: at each step, estimate energy per committed token for every action.
- Selection rule: speculate only if its upper energy margin is below the ordinary action lower margin.
- Fallback: if expected gain is uncertain, use ordinary decoding.
- Four policies are compared: ordinary, best fixed, latency-based and energy-based.

The cost profile (`synthetic_profile.json`) is hand-authored and labelled `SYNTHETIC_DEMO_ONLY`. Its margins are illustrative, not calibrated confidence intervals.

4.3 Selective-Feedback Feasibility Pilot

- Cached greedy drafting and verification with KV-cache rollback.
- Intermediate-exit observation: hidden states are projected through the shared vocabulary head.

- Prefix-validity filter: keeps only predictions made on valid prefixes; rejected-branch and post-EOS outputs are excluded.
- Observation modes: none, one (selected), all and periodic.
- Randomised run order, warmup, and comparison against an independent native Transformers baseline.
- Measurement: request latency, plus NVML GPU joules when available. CPU energy is reported as null, never estimated.

5. System Architecture

5.1 Layered Overview

The system is organised in four layers. Inputs (prompts, cost profile, run configuration) feed a control layer that decides how each token is generated. The execution layer drafts and verifies tokens on the chosen backend, and the output layer measures cost and writes reports.

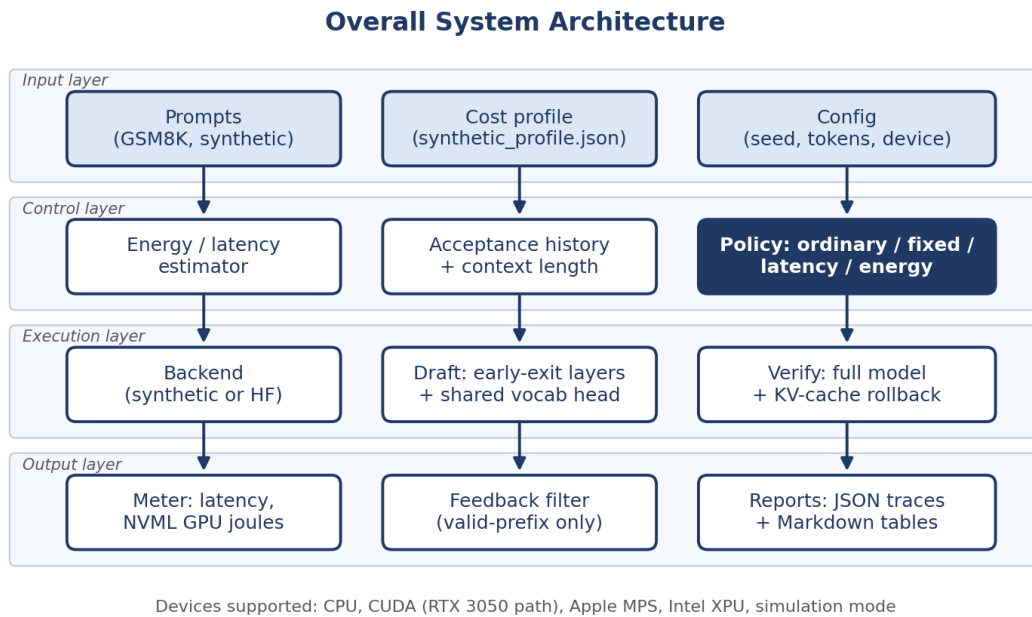


Figure 1: Layered system architecture

5.2 Controller Decision Flow

- At every step the controller reads context length and recent draft acceptance.
- It estimates energy per committed token for ordinary decoding and each draft depth and length pair.
- Speculation is chosen only when its upper energy margin is below the ordinary lower margin; otherwise it falls back.
- Verification always uses the full model, so output is unchanged.

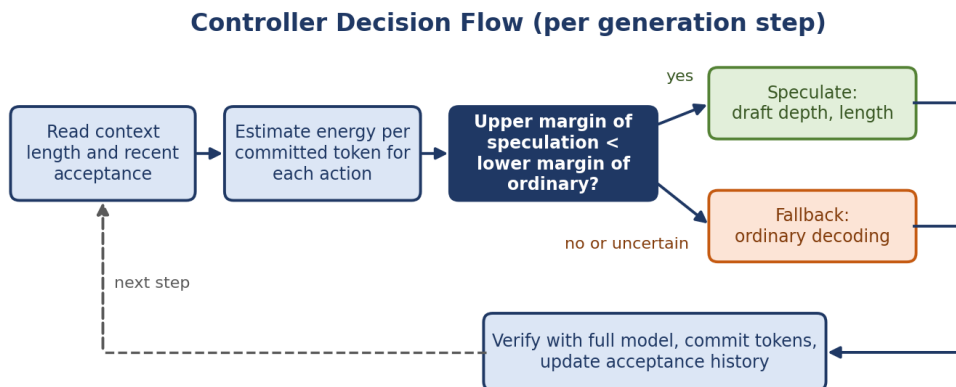
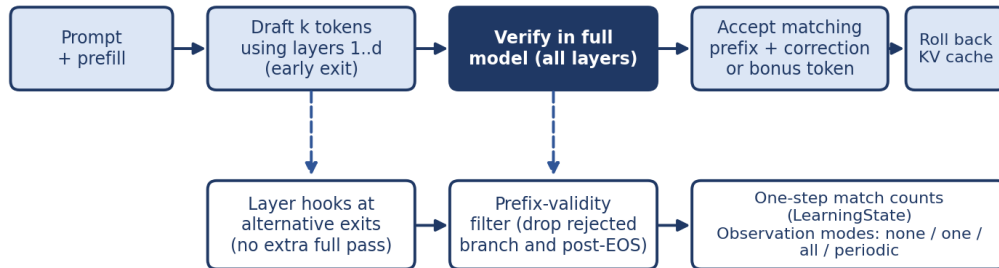


Figure 2: Per-step controller decision flow

5.3 Draft-Verify Engine and Selective Feedback

- Drafting uses early-exit layers and the shared vocabulary head; verification runs all layers.
- Rejected suffixes are discarded and the KV cache is rolled back.
- Hooks read intermediate exits during normal passes, so observation adds no extra full-model pass.
- A validity filter keeps only predictions made on valid prefixes before updating match counts.

Self-Speculative Draft-Verify Engine with Selective Feedback



Greedy decoding only, batch size one. Output must equal native Transformers greedy output.

Figure 3: Draft-verify engine with selective feedback

6. Implementation

Module	Role
prototype/controller.py	Action selection using energy estimates and acceptance history
prototype/backends.py	Synthetic and real execution backends
prototype/devices.py, doctor.py	Device detection (CPU, CUDA, MPS, XPU) and environment check
prototype/experiment.py	Experiment runner and metrics collection
selective_feedback/engine.py	Cached greedy draft and verify loop
selective_feedback/feedback.py	Validity filtering and one-step match statistics
selective_feedback/pilot.py, report.py	Randomised pilot and Markdown/JSON reporting
selective_feedback/tests	Unit tests for correctness, EOS and feedback exclusion

6.1 Technology Stack

Area	Tools
Language and core	Python 3.12, NumPy, psutil, pynvml
Model execution	PyTorch, Hugging Face Transformers (CPU, CUDA, MPS, XPU)
Models and data	SmolLM2-135M, LayerSkip Llama-3.2-1B, Qwen3 (planned), GSM8K, Alpaca
Quantization and inference	PyTorch dynamic INT8 (CPU); llama.cpp planned
Energy measurement	NVML GPU counters; CodeCarbon, HWINFO, powermetrics planned

Reported fields include energy per token, tokens per second, draft acceptance, ordinary-fallback fraction, action counts and whether output matches ordinary decoding.

7. Experimental Setup

- Hardware: Dell Inspiron 13 5330, Intel Core Ultra 7 155H, about 16 GB RAM, Intel Arc integrated graphics.

- No NVIDIA GPU was present, so the planned RTX 3050 CUDA path (scripts/setup_cuda_3050.ps1) was not exercised.
- Models:
 - SmoLM2-135M: pretrained, not exit-trained; used for mechanics only.
 - facebook/layarskip-llama3.2-1B: exit-trained; weights verified by SHA-256.
- Workload: GSM8K test prompts, FP32 on CPU, at most 8 new tokens.

7.1 Evaluation Metrics

- Quality: output match against native greedy decoding (task accuracy or perplexity planned).
- Latency and tokens per second; energy in joules and joules per committed token.
- Draft acceptance rate, ordinary-fallback fraction and controller overhead.
- Peak memory (planned for the full compression study).

8. Results

8.1 Correctness

Model	Prompts	Repeats	Runs	Match native greedy
SmoLM2-135M	2	3	102	102 of 102
LayerSkip 1B	2	1	34	34 of 34

- All five unit tests passed.
- Valid one-step observations: 402 (SmoLM) and 142 (LayerSkip); excluded observations were 576 and 62, which are not counted as failures.

8.2 Median Request Time (seconds, no observation)

Configuration	Draft length	SmoLM2-135M	LayerSkip 1B
Ordinary decoding	0	2.32	3.94
Shallow draft (depth 7 / depth 4)	1	3.07	4.18
Shallow draft (depth 7 / depth 4)	3	3.45	4.54
Deeper draft (depth 15 / depth 8)	1	3.18	4.01
Deeper draft (depth 15 / depth 8)	3	3.70	4.44

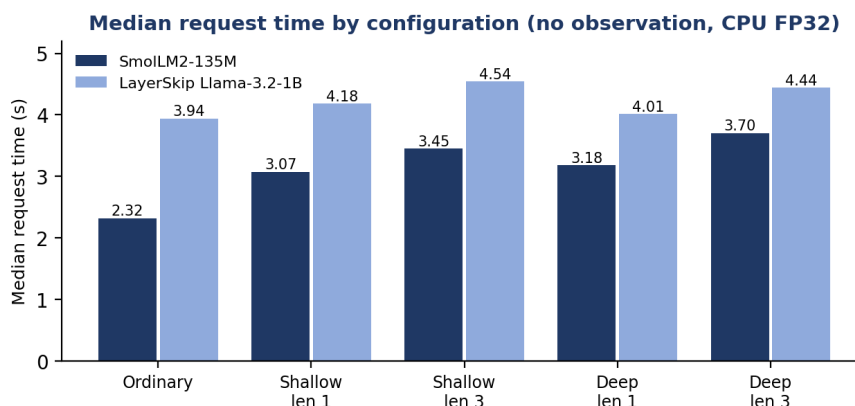


Figure 4: Median request time, CPU FP32, no observation

8.3 Interpretation

- Speculative paths were slower than ordinary decoding in this Python reference at CPU scale.
- Observation overhead is visible but noisy; differences between modes are not conclusive.

- With only two candidate exits, selected and all modes coincide during drafting, so the pilot cannot yet separate them.
- These are smoke tests with 2 prompts and 8 tokens. They confirm correctness, not efficiency.

9. Limitations

- No real energy data: CPU energy is unavailable and GPU counters would exclude CPU controller cost.
- The synthetic prototype uses invented costs; its results are not measurements.
- Quantization, CUDA execution, the adaptive selector and cross-device generalisation are not validated.
- Tiny sample size; the Python implementation is not an optimised serving backend.
- The vendored LayerSkip utility is CC-BY-NC-4.0, which restricts commercial use.

10. Future Work

- Run repeated, sensor-backed energy measurements on an NVIDIA GPU (RTX 3050 path).
- Use larger held-out workloads with separate calibration and test sets.
- Add more candidate exits so selective observation differs from all-feedback.
- Fit energy profiles with calibrated uncertainty and add bootstrap confidence intervals.
- Implement and ablate the adaptive selector against budget-matched baselines.

11. Conclusion

The project has a verified, correct speculative-decoding pipeline and a controller design, but no demonstrated energy saving yet. Real GPU energy measurements are the next decisive step.